From Build Success to Admissible Proof: Evidence-Typed Governance for a Mixed Lean and Executable Formula Corpus

Stephen P. Lutar Jr.
SZL Holdings LLC
ORCID 0009-0001-0110-4173
License: CC BY 4.0

UNRELEASED PREPRINT v0.1.0 - NOT PEER REVIEWED - NO DOI - NOT DEPOSITED

Abstract. Large formalization repositories increasingly mix proof-assistant declarations, executable formulas, tests, manuscript equations, empirical claims, and generated proof obligations. A single status word such as "verified" cannot faithfully summarize these heterogeneous objects. In particular, successful compilation establishes neither the absence of admitted placeholders nor compliance with an axiom policy; passing runtime tests does not establish a theorem; and a declaration name does not establish that two inventories denote the same formula. We present an evidence-typed governance model in which identity, statement binding, source closure, kernel acceptance, axiom policy, runtime testing, empirical support, and refutation are recorded as separate coordinates. The model is applied to a local mixed Lean 4 and executable formula corpus. Deterministic reconciliation finds eight receipt-backed formula families containing nineteen audited declarations, while separately preserving fifteen served-index entries, twenty-one tested Python functions, thirty evidence-ledger items, eighty staged index entries, 179 manuscript blocks, and a 203-record proof-obligation queue. These inventories are non-additive. The queue contains 200 open and three refuted records; it contains no accepted proof trajectory. A fresh whole-project receipt records 13,308 of 13,308 build targets completed, yet source scans record admitted placeholders elsewhere in the repository, illustrating why build success and admissible proof must be separated. We also analyze a machine-checkable counterexample to unconditional uniqueness of an equal-weight geometric-mean aggregator: the maximum aggregator satisfies the weak structural conditions relevant to the counterexample but differs from the geometric mean at `(4,1)`. Negative evidence is therefore retained as a regression artifact rather than recycled as a proof target. The contribution is an evidence schema, a reconciliation method, exact audit tables, and a fail-closed promotion protocol. We make no new mathematical characterization theorem and report no proof-generation experiment. **Keywords:** Lean 4; formal proof; proof artifacts; evidence governance; provenance; reproducibility; axiom audit; source closure; formula corpus; refutation; dataset documentation

This artifact is evidence-gated. Counts, proof classes, execution states, and deployment states are not promoted by implication.

Stephen P. Lutar Jr.

Large formalization repositories increasingly mix proof-assistant declarations, executable formulas, tests, manuscript equations, empirical claims, and generated proof obligations. A single status word such as "verified" cannot faithfully summarize these heterogeneous objects. In particular, successful compilation establishes neither the absence of admitted placeholders nor compliance with an axiom policy; passing runtime tests does not establish a theorem; and a declaration name does not establish that two inventories denote the same formula. We present an evidence-typed governance model in which identity, statement binding, source closure, kernel acceptance, axiom policy, runtime testing, empirical support, and refutation are recorded as separate coordinates. The model is applied to a local mixed Lean 4 and executable formula corpus. Deterministic reconciliation finds eight receipt-backed formula families containing nineteen audited declarations, while separately preserving fifteen served-index entries, twenty-one tested Python functions, thirty evidence-ledger items, eighty staged index entries, 179 manuscript blocks, and a 203-record proof-obligation queue. These inventories are non-additive. The queue contains 200 open and three refuted records; it contains no accepted proof trajectory. A fresh whole-project receipt records 13,308 of 13,308 build targets completed, yet source scans record admitted placeholders elsewhere in the repository, illustrating why build success and admissible proof must be separated. We also analyze a machine-checkable counterexample to unconditional uniqueness of an equal-weight geometric-mean aggregator: the maximum aggregator satisfies the weak structural conditions relevant to the counterexample but differs from the geometric mean at `(4,1)`. Negative evidence is therefore retained as a regression artifact rather than recycled as a proof target. The contribution is an evidence schema, a reconciliation method, exact audit tables, and a fail-closed promotion protocol. We make no new mathematical characterization theorem and report no proof-generation experiment.

Keywords: Lean 4; formal proof; proof artifacts; evidence governance; provenance; reproducibility; axiom audit; source closure; formula corpus; refutation; dataset documentation

1. Introduction

The statement that a formula is "verified" can refer to several materially different events. A function may have passed unit tests. A declaration with the expected name may be present in a source tree. A file may elaborate because an admitted placeholder was converted into an axiom. A whole project may build at a pinned commit. A theorem may be accepted by a proof assistant's kernel, but depend on axioms outside an intended policy. A physical formula may be dimensionally consistent without having empirical support. A manuscript equation may be well typeset but not bound to any formal declaration. Finally, a conjecture may be disproved by a small counterexample even while an old source file still contains the conjecture as an admitted theorem.

Collapsing these events into one Boolean creates an epistemic type error. The error is especially likely in mixed repositories, where formal sources coexist with application code, registries, tests, prose, data, and legacy counters. A build dashboard encourages aggregation because it offers one conspicuous green signal. A formula registry encourages aggregation because it offers one list of names. A publication outline encourages aggregation because it numbers displayed equations. None of those interfaces establishes that the underlying objects share a statement, proof regime, dependency closure, or interpretation.

This paper studies that problem through an audit of a local corpus containing Lean 4 declarations, Mathlib dependencies, Python implementations, served formula metadata, manuscript blocks, and a proof-obligation dataset. The corpus resists a simple headline count. It contains genuine kernel-checked work, executable behavior, open obligations, and counterexamples. It also contains historical identifier reuse, source counters, and a successful full build that would support an exaggerated claim if used without its boundary conditions.

Our central position is that evidence must be typed before it is counted. "Typed" here does not mean that all research evidence is reduced to a theorem-prover type. It means that every status assertion carries a declared kind and scope. Presence, source closure, kernel acceptance, axiom compliance, testing, measurement, and refutation are independent facts. Some combine into an admission decision; none is silently substituted for another. A receipt is valuable only when it binds an exact statement and artifact identity to an exact toolchain and evidence surface.

The paper addresses four research questions:

- **RQ1:** Can a multidimensional evidence model reconcile heterogeneous formula inventories without summing or promoting them?
- **RQ2:** What additional evidence separates successful whole-project compilation from a claim that particular declarations are admissibly proved?
- **RQ3:** How should refuted statements and semantic identifier collisions be represented so that automation fails closed?
- **RQ4:** What reproducible artifact set is required to turn a local audit into a reviewable proof-admission dataset?

The contributions are methodological and empirical within the audited scope:

1. We define an evidence record and a product-style evidence order that keep artifact identity, statement binding, source closure, kernel acceptance, axiom policy, tests, empirical evidence, and counterevidence distinct.
2. We give a deterministic reconciliation procedure for mixed Lean and executable corpora, including rules for non-additive counts, historical pin mismatch, and formula-ID collisions.
3. We report exact local results: eight locked families, nineteen audited declarations, a successful 13,308-target build, and a 203-record obligation queue with 200 open and three refuted records.
4. We use the `maxAgg_ne_Lambda` counterexample as a case study in preserving negative evidence and narrowing a false unconditional theorem to a conditional, explicitly assumption-bearing frontier.
5. We specify a fail-closed promotion protocol and artifact checklist intended to make every public proof claim independently reviewable.

These contributions are deliberately narrower than a theorem-count paper or a neural theorem-proving paper. We do not report 180 or 200 proved formulas. We do not treat 13,308 build targets as theorems. We do not report accepted proof trajectories, training runs, proof-search success rates, user studies, or production deployment. We do not claim a new uniqueness theorem for the geometric mean. The strongest headline supported by the locked local receipt is eight formula families containing nineteen audited declarations, under the exact encoded statements and disclosed axiom surface.

2. Background and problem setting

2.1 Lean, Mathlib, and layers of acceptance

Lean 4 is both an interactive theorem prover and a functional programming language. Its elaborator transforms user-facing syntax and tactics into terms checked against Lean's logical foundations; Mathlib supplies a large, community-developed body of formal mathematics and automation [1, 2]. These layers enable sophisticated formalization but produce distinct facts that are easily conflated.

First, a declaration may be present as text. Second, Lean may elaborate its module. Third, the project may build. Fourth, a selected declaration may have no unresolved placeholder in its intended dependency closure. Fifth, inspection of its axioms may show only the foundations permitted by a project policy. The first three facts are useful, but a strict claim of admissible proof requires the latter facts as well. In this manuscript, "kernel accepted" refers to acceptance by the pinned Lean environment for the exact declaration and source identity. "Admissibly proved" adds source-closure and axiom-policy requirements defined below. It does not mean philosophically absolute truth, empirical validity, security of an implementation, or correctness of an informal interpretation.

An admitted `sorry` illustrates the distinction. It enables continued formal development by supplying a placeholder proof. The surrounding file may compile, and a whole-project build may remain green, but the relevant proposition is not established from the disclosed allowed foundation in the same sense as a closed proof term. A source-wide `sorry` count is also not a theorem count: comments, declarations, dependencies, and intentional axioms

require contextual classification. Consequently, neither "build passed" nor "the source contains N declarations" is a valid substitute for per-claim admission evidence.

2.2 Mixed formula corpora

The audited estate contains several kinds of record:

- Lean declarations and source modules;
- a receipt enumerating selected declarations and printed axiom dependencies;
- Python functions exercised by runtime tests;
- a served registry that maps public endpoints to declaration names or experimental formulas;
- symbolic, dimensional, definitional, and empirical evidence records;
- a staged advisory theorem index;
- manuscript equations and formal blocks;
- and a JSON Lines queue describing work to be formalized, rebound, measured, or preserved as a counterexample.

Each inventory has a valid local purpose. The mistake is to assume that its unit of counting is interchangeable with every other unit. One formula family can contain several declarations. One executable function can implement several branches. One manuscript block can refer to no current declaration. A queue item describes work, not a completed proof. A served entry may prove only that a name exists in a corpus. The same short identifier may even refer to different semantics in two sources.

2.3 Proof, test, support, and refutation

We use the following ordinary-language distinctions throughout:

- **PROVED** is reserved for an exact formal statement that passes the declared admission policy at pinned identities.
- **TESTED** records execution against a declared test set. It supports behavioral claims within that scope, not universal mathematical claims.
- **SUPPORTED** records relevant evidence that is insufficient for PROVED, for example source presence, symbolic agreement, dimensional checking, literature, or a definition.
- **SUPPORTED_MODEL** additionally emphasizes that the theorem concerns an abstraction rather than the full deployed system suggested by a name.
- **EMPIRICAL** requires observations, calibration or dataset identity, uncertainty, and an analysis protocol.
- **OPEN** names an unclosed obligation without implying that the proposition is true.
- **REFUTED** records a counterexample or contradiction to the exact target.
- **EXPERIMENTAL** identifies an implementation or formula that has not crossed the formal or empirical admission gate.
- **UNKNOWN** means that the available record cannot justify a stronger type.

These labels are not a single ladder. REFUTED is not "below" OPEN in a confidence score: it is stronger negative information. TESTED is not one step below PROVED; it lives on a different evidence axis. A deterministic runtime implementation of a false general conjecture can be thoroughly tested. Conversely, a noncomputable theorem can be proved without a runtime test.

2.4 Aggregation functions and the Lambda case

One corpus thread concerns the equal-weight geometric mean on nonnegative reals:

```
Lambda_k(x) = (prod_{i=1}^{k}x_i)^{1/k}, k>0.
```

The local Lean definition uses `NNReal` and finite axes. The accompanying structure collects monotonicity (A1), positive homogeneity (A2), diagonal normalization (A3), an upper bound by the maximum axis (A4), and permutation invariance (A5). These are plausible properties of an aggregator, but plausibility is not uniqueness. Characterization results for aggregation and quasi-arithmetic means depend sensitively on assumptions such as separability, associativity, bisymmetry, compatibility, strict monotonicity, or regularity [10-12]. The local corpus is therefore an instructive test: if a weak axiom set admits another aggregator, the proper output is a retained counterexample and a narrowed conditional theorem, not a generated proof of the old target.

3. An evidence lattice for formal and executable claims

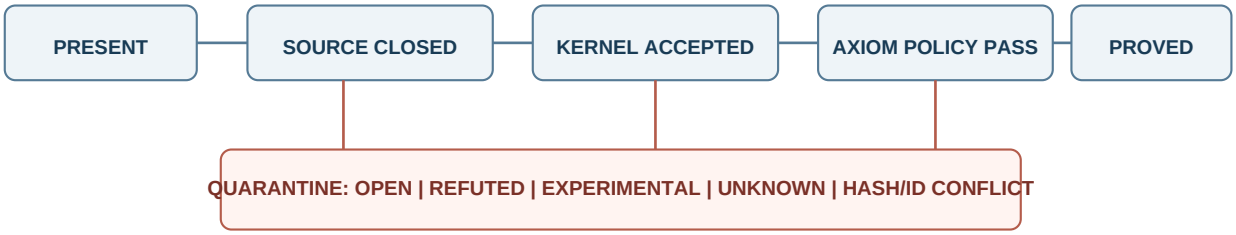


Figure 1. Evidence admission and non-promoting quarantine paths.

3.1 Records, artifacts, and statement identity

Let R be a set of governed claim records. A record r in R is a tuple

`r=(u,s,a,d,t,p,e,n),`

where u is a stable record identifier; s is the normalized statement and interpretation boundary; a identifies source artifacts; d identifies dependencies and toolchain; t is a vector of formal evidence; p is a policy; e is a vector of executable or empirical evidence; and n is negative evidence. This is a governance definition, not a newly formalized Lean theorem.

For a byte sequence b, define its local artifact identity as

`aid(b)=(path,SHA256(b),bytes).`

A version-control commit is recorded in addition to, not instead of, the content digest. The commit establishes repository lineage; the digest checks the exact file. A mutable branch name does neither. A container tag is likewise insufficient when a content-addressed image digest is available.

A statement binding is a tuple

`bind(r)=(u,s,q,m,c,h),`

where q is a fully qualified declaration name, m a module path, c a source commit, and h a source-file digest. A short identifier such as `F1` is an alias, not a binding. A binding is **exact** only if the natural-language statement, encoded theorem type, interpretation boundary, module, declaration, commit, and digest agree. If two records share an alias but differ in statement or binding, the join result is `COLLISION`, not the union of their evidence.

3.2 Formal evidence coordinates

For each record we track at least five formal coordinates:

1. I, **identity**, ranges over `UNRESOLVED`, `PATH_ONLY`, `COMMIT_PINNED`, and `CONTENT_PINNED`.
2. B, **binding**, ranges over `UNSPECIFIED`, `CANDIDATE`, `DECLARATION_PRESENT`, and `STATEMENT_EXACT`.
3. S, **source closure**, ranges over `NOT_SCANNED`, `PLACEHOLDER_PRESENT`, `TARGET_CLEAN`, and `CLOSURE_CLEAN`.

4. K, **kernel acceptance**, ranges over `NOT_RUN`, `FAILED`, `MODULE_ACCEPTED`, and `DECLARATION_ACCEPTED`.

5. A, **axiom policy**, ranges over `NOT_INSPECTED`, `OUTSIDE_POLICY`, and `POLICY_PASS`.

The order within each positive coordinate represents additional evidence, except that failure states are absorbing side branches rather than weak successes. The product order compares two compatible records coordinatewise: x <= y only when each positive coordinate of y is at least as informative as the corresponding coordinate of x, and neither record carries conflicting identity or refutation evidence. Most pairs are incomparable. For example, a heavily tested Python implementation and a source-present Lean theorem need not dominate one another.

We define the formal admission predicate for policy P as

```
Admissible_P(r) <=> {} I(r)=CONTENT_PINNED and
B(r)=STATEMENT_EXACT and
S(r)=CLOSURE_CLEAN and
K(r)=DECLARATION_ACCEPTED and
A_P(r)=POLICY_PASS and
N(r)!=REFUTED.
```

The locked receipt used in this audit gives exact source identity, a clean target module scan, successful Lean processing, and per-declaration axiom output under its stated scope. We therefore use its own word "proved" only within that receipt scope. For a future public artifact, `CLOSURE_CLEAN` should be established by an explicit dependency-closure procedure rather than inferred from a target-file scan. This distinction prevents our abstract admission definition from overstating what the current local receipt demonstrates.

3.3 Executable and empirical coordinates

The executable vector contains at least:

- X: executable identity and environment;

- T: test protocol, selected cases, and result;

- M: measurements, dataset, calibration, baseline, and uncertainty;

- O: operational or deployment observation.

These axes deliberately do not feed the formal admission predicate. We record

```
Tested(r) <=> X(r) is pinned and the declared test protocol passes,
```

and

```
EmpiricallySupported(r) <=> M(r) identifies observations, method, uncertainty, and applicable scope.
```

Neither predicate implies Admissible_P(r). Conversely, a proof about an encoded model does not establish that a deployed implementation conforms to that model or that a natural phenomenon follows it. A bridge theorem or conformance test can link evidence coordinates, but the bridge itself must be an admitted record with an exact interface and scope.

3.4 Negative evidence

Negative evidence is represented explicitly rather than encoded as a low score. A refutation record contains the exact target statement, a witness or derivation, the environment in which it was checked, and the target revision. Define

```
Refuted(r) <=> exists w. CheckCounterexample(s(r),w)=PASS.
```

When `Refuted(r)` holds, the same normalized statement cannot be promoted to PROVED. A later theorem can be admitted only if its statement or assumptions change and it receives a new stable identity. The refutation remains attached to the old target as a regression test. This "negative monotonicity" rule is central: additional source files cannot erase a valid counterexample by relabeling it OPEN.

3.5 Non-promotion invariants

The governance model enforces the following invariants:

- **Presence is not proof:** `DECLARATION_PRESENT` does not imply kernel acceptance, source closure, or policy pass.
- **Build is not closure:** a successful project build does not imply that every declaration is free of admitted placeholders.
- **Tests are not theorems:** passing a finite runtime suite does not prove a universal proposition.
- **Models are not deployments:** an admitted theorem about an abstract queue, code, budget, or additive module does not automatically apply to a production system.
- **Names are not semantics:** equal aliases do not authorize a join when statement or provenance differs.
- **Counts are scoped:** only records with the same unit, scope, and state may be aggregated.
- **Refutations persist:** a counterexample prevents promotion of the exact target and is retained after a replacement statement is introduced.
- **Unknown remains unknown:** missing source, environment, measurement, or receipt yields an explicit unavailable state, never positive credit.

The lattice-like product prevents compensation: many tests cannot numerically outweigh one unresolved formal coordinate.

3.6 Receipts and validation

A proof receipt should be machine-readable and minimally include:

```
schema, observed_at, verdict, scope
repository, commit, source_path, source_sha256
Lean_version, Mathlib_commit, platform, environment_digest
command, exit_code, declaration_name, normalized_statement_digest
placeholder_scan_scope, placeholder_count
printed_axioms, allowed_axioms, axiom_policy_result
claim_boundary, reviewer_or_signature_state
```

Receipt validation is conjunctive: recompute digests, resolve pins, rerun the target, scan the declared closure, compare printed axioms with the allowlist, and check that the interpretation does not exceed the theorem type. A deterministic digest supports replay but is not a signature, author identity, peer review, or independent replication.

4. Corpus and reconciliation methodology

4.1 Audit scope and mutation boundary

The audit was conducted over local artifacts available on 12 July 2026. It did not modify production code, remote repositories, deployed services, or external data. The primary sources were a formula-readiness ledger, a JSON Lines proof-obligation queue, an eight-family Lean receipt, a fresh whole-project build receipt, an A11oy served index and registry guard, a canonical Python formula registry, selected Lean modules for the Lambda and Ouroboros cases, and a prior formula/proof frontier audit.

The unit of analysis is always reported. A "family" is not a declaration. A build target is not a theorem. A Python function is not a formula family. A manuscript block is not a proof obligation unless an explicit record binds it. This seems elementary, but most misleading totals arise by violating exactly this rule.

4.2 Source artifacts

Table 1 lists the core frozen inputs. Digests were recomputed locally. Paths are relative to the accompanying preprint archive. Exact path-safe files are copied byte-for-byte; files containing developer-machine or stale workspace paths are sanitized, with both source and packaged digests recorded in `SOURCE_TRANSFORMATIONS.json`.

Table 1. Core audit artifacts.

Artifact	Bytes	SHA-256	Role
`evidence/paper1/formula-proof-frontier-audit.sanitized.md`	14,709 source bytes	`d6d7aa314d3983e527597e979f2ef9aa3a3b3275dc0fbd49afe15c1756ce72c9` (source)	Prior consolidated audit and claim boundaries
`evidence/paper1/lutar-lean-full-build-receipt.json`	1,054	`50af3d31ec20efc9a696a71130958c653e5fb125174567ac6cebf66bd954b875`	Whole-project build evidence; exact copy
`evidence/paper1/lean-proved-formulas-receipt.json`	3,239	`7ace9bf5d7cdbad19ec4f9c17dc44ff24697050e96dd245713faf8f83407d25f`	Locked eight-family/19-declaration evidence; exact copy
`data/proof_obligations_v1.jsonl`	106,642	`fc344205065147255093a6d947749abc9a20f1251c02c461498291cb8cc20952`	203-record work queue; exact copy
`data/formula_readiness_ledger.json`	158,697 source bytes	`a3bed18056859153475f3bb8ca7263fa8439680e2555e95f7eacf907d66ebf55` (source)	Path-sanitized formula/evidence inventory
`evidence/shared/lake-mathlib-octave-operational-contract.sanitized.json`	1,916 source bytes	`589a466436558ad3ca5f58d9a6abd9c2f7f0d522da6df7e7753cd3ba4e806583` (source)	Source-availability and identity constraints

The full build receipt binds Lutar-Lean commit `85dee499b72cc71f49907c70e62018ab85dba01c`, Lean `v4.18.0`, and Mathlib commit `aa936c36e8484abd300577139faf8e945850831a`. It reports a fresh Ubuntu Docker container using official `elan`, the commands `lake exe cache get` and `lake build`, exit code zero, 13,308 of 13,308 targets completed, final target `check`, and 21 passing guard tests with zero failures.

The selected-declaration receipt binds the same working-tree base, Lean 4.18.0, the module `Lutar/Puriq/Formulas/ProvedFormulas.lean`, and source-file SHA-256 `acce5d750702cff89c0f84a9cc0a5d7ff22264d76d5229e1b47d3613b7ab77a1`. It reports zero non-comment `sorry` in that module and per-declaration axiom sets limited to `propext`, `Quot.sound`, or no axioms. Its command is `lean Lutar/Puriq/Formulas/ProvedFormulas.lean` in a container tagged `szl-lean-audit:20260712`. A public reproduction should replace that mutable tag with an image digest.

4.3 Reconciliation algorithm

The audit applies the following deterministic stages:

1. **Freeze inputs.** Record path, byte length, SHA-256, repository commit when available, and toolchain pins.
2. **Normalize units.** Tag every inventory row as family, declaration, executable function, served entry, evidence item, staged entry, manuscript block, or obligation.
3. **Normalize evidence.** Map source-specific labels to the evidence coordinates without upgrading them. For example, "declaration-backed" becomes declaration presence, not PROVED.
4. **Resolve bindings.** Require fully qualified names and semantic statements. Mark missing or changed declarations as binding repairs.
5. **Detect collisions.** Compare shared aliases across inventories. Quarantine any alias that maps to inconsistent semantics.
6. **Validate positive formal claims.** Check exact receipt scope, commit, source digest, target execution, placeholder scan, and axiom allowlist.

7. **Preserve negative claims.** Retain counterexamples and map false targets to REFUTED rather than sending them to proof generation.
8. **Count within partitions.** Group only by common inventory, unit, and status. Never sum across the resulting tables.
9. **Emit obligations.** Assign incomplete work to one of four task classes: formalize and prove, repair exact Lean binding, discharge or measure, or preserve counterexample.

In pseudocode:

```
for artifact in inputs:
    verify_sha256(artifact)
    parse_records(artifact)

for record in records:
    record.unit = classify_unit(record)
    record.evidence = type_without_promotion(record.source_status)
    record.binding = resolve_exact_statement_and_declaration(record)
    if alias_collision(record):
        record.binding = COLLISION
        quarantine(record)
    if has_counterexample(record):
        record.disposition = REFUTED
        preserve_regression(record)
    elif claims_proved(record):
        validate_receipt_conjunctively(record)

report(group_by(inventory, unit, disposition))
assert no_cross_inventory_sum(report)
```

4.4 Historical pin reconciliation

The selected-declaration receipt contains both working-tree base `85dee499...` and older doctrine pin metadata `c7c0ba17c2eaec60ad38ea9172b4a0d9ca0b582f`. Direct inspection found that the older `c7` checkout does not contain the selected `ProvedFormulas.lean` module or its companion `PROVEN_FORMULAS.md`. The receipt itself therefore states that reconciliation is required and prohibits attributing the result to `c7`.

We therefore use the source digest and `85dee499...` base as evidence identity. The older value remains provenance metadata, not a build source; historical doctrine and executable evidence are separate fields.

4.5 Formula-ID collision detection

Eight public family labels occur with conflicting semantics in two inventories. Table 2 shows the observed collision. The left column describes the Brain-side use; the right column describes the locked receipt's declarations. These rows must not be joined by `F*` alone.

Table 2. Quarantined formula-ID collisions.

Alias	Brain-side description	Locked Lean receipt description
F1	Euler-Khipu DAG	replay fold determinism and trace binding
F4	Gauss-Yuyay	Khipu DAG acyclicity under a backward-edge invariant
F7	inverse-square/zeta	Chaski list-queue FIFO properties
F11	frustum	Ayni integer reciprocity/conservation
F12	CRT schedule	additive Kuramoto encoding
F18	Kolmogorov	Reed-Solomon parity arithmetic
F19	Turing fuel	additive budget scaffold named for Bekenstein
F22	Feynman	Khipu `List.range` emission monotonicity

The required remedy is a signed-or-explicitly-unsigned crosswalk with a stable global identifier, local alias, semantic statement, domain and codomain, Lean declaration, module, commit, file digest, and receipt digest. Until that crosswalk exists, label-based evidence transfer fails closed. This collision also prevents a false impression that the eight receipt-backed families validate the eight Brain-side formulas of the same labels.

4.6 Queue classification

Every row of `proof_obligations_v1.jsonl` uses schema `szl.proof-obligation/v1`, a stable `id`, source scope, disposition, task, evidence note, and a common admission rule requiring a pinned Lean build, zero `sorry`/`admit`, and an explicit axiom allowlist. Optional Lean file, theorem, and statement fields may be null when specification or binding work remains.

The queue is a planning dataset, not proof data. `FORMALIZE_AND_PROVE` names 131 candidate formalization tasks. `REPAIR_EXACT_LEAN_BINDING` names 63 metadata/binding repairs. `DISCHARGE_OR_MEASURE` names seven records whose claims depend on a named dataset or observation. `PRESERVE_COUNTEREXAMPLE` names two tasks whose successful outcome is continued refutation coverage. Status counts are computed from the `disposition` field, not a nonexistent `status` field.

5. The Lambda refutation case

5.1 Exact local formulation

The local Lean module defines an aggregator over `Axes k = Fin k -> NNReal` and defines

```
noncomputable def Lambda (k : N) (x : Axes k) : NNReal :=
  if hk : k = 0 then 0
  else ((Finset.univ : Finset (Fin k)).prod x) ^ ((1 : R) / (k : R))
```

For positive `k`, the branch is the ordinary equal-weight geometric mean. The `LutarAxioms` structure contains:

- A1: coordinatewise monotonicity;
- A2: positive homogeneity of degree one;
- A3: positive arity and diagonal normalization `Phi (fun _ => c) = c`;
- A4: an upper bound by the maximum coordinate;
- A5: invariance under permutations of axes.

The Round 13 source proves that `Lambda` satisfies these axioms and defines a factorization predicate

```
Factors(Phi,alpha) <=>
forall x, Phi(x)=prod_i x_i^{alpha_i}.
```

It then provides `lambda_unique_of_factors`: given positive arity, the five axioms, and this explicit weighted-product factorization, the exponents are forced to the equal value and Phi=Lambda_k. That is a conditional source theorem. It is not included in the locked nineteen-declaration receipt, and this manuscript does not independently promote it.

5.2 Counterexample

For arity two, the source defines

```
maxAgg(x)=max(x_0,x_1).
```

The module contains machine-checked lemmas for permutation invariance, diagonal normalization, and positive homogeneity, and its commentary identifies monotonicity and the maximum bound as the remaining immediate order properties of the maximum operation. The decisive theorem is `Round13.maxAgg_ne_Lambda`. It evaluates both functions at the vector `(4,1)`:

```
maxAgg(4,1)=4,
Lambda_2(4,1)=(4* 1)^{1/2}=2.
```

Hence `maxAgg != Lambda 2`. Because the maximum aggregator meets the weak shape constraints used by the unconditional target, A1-A5 do not uniquely characterize the equal-weight geometric mean. The source also notes minimum as another conceptual counterexample, although `maxAgg_ne_Lambda` is the explicit machine-checkable witness audited here.

The old unconditional declaration `lambda_unique` remains in the source with one tagged `sorry` and commentary that the statement is false without an added factorization-producing assumption such as bisymmetry or associativity. An older `Uniqueness.lean` path contains two tracked `sorry` obligations. These placeholders cannot be closed honestly by more proof search because the target has a counterexample.

5.3 Governance consequence

The correct transition is

```
unconditional A1-A5 uniqueness
    -> REFUTED (retain maxAgg_ne_Lambda)

factorization/bisymmetry-strengthened statement
    -> CONDITIONAL_SOURCE_PRESENT
    -> dedicated receipt required before PROVED
```

The record identity must change when the hypotheses change. Otherwise, a system could overwrite a refuted statement with a stronger true one and misleadingly report that the original conjecture was proved. The counterexample should be executed in continuous integration as a negative regression: any future schema, generator, or registry that tries to promote the weak target must fail.

This result is not surprising relative to the mathematical literature. Classical work on synthesizing ratio judgments explicitly studies additional properties including separability, associativity or bisymmetry, cancellativity, consensus, reciprocity, and homogeneity [10]. Modern characterizations of row-geometric-mean rankings and quasi-arithmetic means likewise state the assumptions that drive uniqueness [11, 12]. The local error was therefore not a new mathematical paradox; it was a governance failure to keep a theorem's assumption surface attached to its status.

5.4 Broader lesson from refutation

Proof automation often treats an unfinished theorem as a search problem. Governance must first test whether it is a truth-candidate under the current assumptions. A small refutation can prevent repeated search, sharpen the specification, and document the boundary between false and potentially true formulations.

REFUTED records should retain target, witness, checking environment, and replacement route. The current corpus contains zero accepted proof trajectories, so we make no claim about training value or model performance.

6. Implementation

6.1 Machine-readable records

The present implementation is artifact-oriented. JSON records carry a schema identifier and preserve source-native fields; Markdown is generated or checked against those records for human review. The proof-obligation schema already demonstrates the minimum pattern:

```
{
  "schema": "szl.proof-obligation/v1",
  "id": "<stable scope-qualified identifier>",
  "source_scope": "<originating inventory>",
  "disposition": "OPEN | REFUTED",
  "task": "FORMALIZE_AND_PROVE | REPAIR_EXACT_LEAN_BINDING | DISCHARGE_OR_MEASURE | PRESERVE_COUNTEREXAMPLE",
  "statement": null,
  "lean_file": null,
  "lean_theorem": null,
  "evidence": "<current evidence and boundary>",
  "admission_rule": "pinned Lean build; zero sorry/admit; explicit axiom allowlist"
}
```

Null binding fields are meaningful. They state that the record has not yet acquired an exact formal target. Converting null to a guessed theorem name would make the dataset look more complete while reducing its reliability. The future crosswalk schema should likewise distinguish `null`, `unavailable`, `collision`, and `not_applicable` rather than encoding all four as an empty string.

Each positive receipt is paired with a claim boundary. For example, the F18 declarations establish parity arithmetic and an erasure-count inequality in an encoded RS(10,6) setting; they do not establish correctness of a Reed-Solomon decoder. The boundary is not decorative prose. It is part of the record that a public registry must display alongside `PROVED`.

6.2 Admission state machine

The intended state machine is:

```
UNSPECIFIED
    |
    v
PRESENT -----> COLLISION / SOURCE_UNAVAILABLE
    |
    v
STATEMENT_BOUND -----> REFUTED
    |
    v
SOURCE_CLOSED -----> PLACEHOLDER_PRESENT
    |
    v
KERNEL_ACCEPTED -----> BUILD_FAILED
    |
    v
AXIOM_POLICY_PASS -----> AXIOM_POLICY_FAIL
    |
    v
PROVED_WITHIN_SCOPE
```

TESTED, EMPIRICAL, EXPERIMENTAL, and DEPLOYED are orthogonal annotations, not intermediate nodes on this path. A runtime test failure may block an implementation release without changing a theorem's proof status. A theorem may remain proved while an empirical interpretation is rejected. A source-unavailable record can retain earlier evidence, but it cannot be refreshed or promoted.

Every transition requires a witness artifact. `PRESENT` requires a resolved path and digest. `STATEMENT_BOUND` requires the normalized theorem type and interpretation. `SOURCE_CLOSED` requires a scan over the declared closure. `KERNEL_ACCEPTED` requires a pinned command and successful result. `AXIOM_POLICY_PASS` requires actual axiom output compared with a versioned allowlist. The final state is derived, never written by hand as an independent Boolean.

6.3 Counterexample-first workflow

For OPEN formal records, the workflow begins with specification review and counterexample search. Finite or decidable statements can often be explored by enumeration, `decide`, `native_decide`, simplification, or a small executable oracle. Algebraic statements can be checked at boundary values and under violated side conditions. State-machine statements should be tested against empty traces, repeated events, and invariant-breaking transitions. Analytic claims should be inspected for missing completeness, measurability, positivity, integrability, or contraction hypotheses.

Only after this phase should proof search begin. Candidate proofs are compiled; nonzero exits, placeholders, unexpected axioms, source drift, and duplicate or vacuous targets are rejected. Parallel generation can increase the number of candidates, but candidates do not vote a proposition true. Kernel acceptance and policy checks remain deterministic gates.

6.4 Build and axiom reporting

The whole-project receipt and selected-declaration receipt play different roles. The former establishes that a pinned repository and dependency revision built successfully in a fresh environment. It is valuable for reproducibility and integration. The latter names exactly nineteen declarations, their target source, target placeholder count, and their axiom surfaces. It is the evidence used for the locked proof count.

For a public artifact, the build job should emit a whole-project environment receipt, a per-declaration theorem-type and axiom report, and a dependency-closure placeholder report. They are joined on source digest and toolchain identity. Commit or digest mismatch fails validation; an unexpected axiom yields `AXIOM_POLICY_FAIL`; an interpretation broader than the theorem type yields a binding error.

6.5 Count regeneration

The counts in this preprint were regenerated from the frozen JSON and JSON Lines inputs with deterministic grouping. The critical implementation rule is to group the queue by `disposition`, because its status field is named `disposition`, and by `task`. The accompanying standalone validator checks expected digests before parsing and emits a machine-readable result; a future release may add a smaller count-only utility. Hard-coded totals belong only in regression assertions tied to a dataset version.

A count report records input digest, parser version, grouping key, unit, partition, and total. This makes "203" reproducible as obligation records and "19" as declaration entries, while preventing a presentation layer from summing them.

7. Results

7.1 Inventory reconciliation

Table 3 gives the principal result. The inventories coexist, but their totals are not additive. Each row answers a different question.

Table 3. Reconciled inventory counts and honest interpretations.

Scope	Exact count	Partition	Evidence established
Locked formula families	8	all `PROVED` within receipt scope	Family grouping for the nineteen audited declarations
Locked Lean declarations	19	all receipt-backed	Exact encoded theorem statements with disclosed axiom surface
Served A11oy index	15	7 declaration-backed; 8 experimental; 0 unbacked	Name/corpus presence for the seven, not proof admission
Canonical Python registry	21	all `TESTED`	Executable behavior under its runtime tests
Evidence ledger	30	19 tested; 4 supported; 1 refuted; 6 open	Heterogeneous symbolic, dimensional, definitional, or empirical evidence
Staged advisory index	80	17 supported; 63 open	Advisory historical bindings
Manuscript CSV	179	46 supported; 2 refuted; 131 open	Versioned manuscript blocks, not declarations
Proof-obligation queue	203	200 open; 3 refuted	Bounded work items, not trajectories or proofs

This table resolves several superficially conflicting numbers without choosing one "true corpus size." There are eight locked families and nineteen locked declarations because the units differ. There are twenty-one Python functions because executable registry boundaries differ. There are 179 manuscript blocks because prose segmentation differs. The correct total depends on the question, and no row answers "how many formulas exist" without a scope definition.

The audit found no artifact supporting an exact total of 180 proved formulas. It therefore rejects `180 proved` as unsourced. It also rejects `200 proved`: 200 is the number of OPEN rows in the obligation queue. The number 203 is the queue size, not the number of proof trajectories. The number of accepted local proof trajectories is zero.

7.2 Whole-build result and source counters

The fresh build receipt is a strong integration result: exit code zero, 13,308 of 13,308 targets completed, and 21 of 21 guard tests passed at the pinned Lutar-Lean and Mathlib commits. It is not a result that 13,308 theorems were proved. Build targets can include modules, generated objects, and checks, and a successful build can include declarations that depend on admitted placeholders or named axioms.

Table 4 shows why source counters cannot be promoted.

Table 4. Source counters, reported as counters rather than theorem totals.

Snapshot	Declaration counter	Unique named-axiom counter	Non-comment `sorry` counter
Locked-v11 source accounting	749	14	149
Audited current-main source accounting	1,401	25	270

These figures are not used to infer how many statements are valid, invalid, closed, or public-facing. They do demonstrate that the repository-wide build receipt cannot by itself establish zero admitted placeholders. Differences between snapshots can arise from growth, changed counting rules, or source drift and would require a dedicated lineage analysis before trend claims were appropriate.

7.3 Locked declaration set

Table 5 reports every declaration in the selected receipt. "None" means that the receipt reported an empty axiom list for that declaration. The model boundary is summarized at family level but applies to the exact theorem type, not merely the label.

Table 5. Nineteen locked declarations and disclosed axiom surface.

Family	Lean declaration	Reported axioms	Interpretation boundary
F1	`f1_replay_fold_deterministic`	none	Pure transition/fold determinism; no cryptographic security
F1	`f1_replay_fold_eq_trace_last`	`propext`	Encoded trace binding only
F4	`f4_khipu_dag_acyclic_preserved`	`propext`	Natural-number edge-list model under explicit backward-edge invariant
F7	`f7_chaski_fifo_order`	`propext`	Encoded list-queue FIFO
F7	`f7_chaski_fifo_positional`	`propext`	Encoded positional list property
F7	`f7_chaski_drain_eq`	`propext`	Encoded queue drain equality
F11	`f11_ayni_reciprocity_conservation`	`propext`	Integer credit/debit conservation; no equilibrium claim
F11	`f11_tit_for_tat_parity`	`propext`	Mirrored-delta parity only
F12	`f12_kuramoto_additive`	none	Additive encoding; no nonlinear synchronization theorem
F12	`f12_kuramoto_superposition`	`propext`	Linear superposition only
F18	`f18_reed_solomon_parity_count`	none	RS(10,6) parity arithmetic; no decoder correctness
F18	`f18_erasure_tolerance`	`propext`, `Quot.sound`	Erasure-count inequality only
F19	`f19_bekenstein_additive`	none	Additive budget scaffold; not the physical Bekenstein bound

Family	Lean declaration	Reported axioms	Interpretation boundary
F19	`f19_budget_monotone`	none	Encoded budget monotonicity
F19	`f19_budget_total_cons`	`propext`	Encoded total conservation
F19	`f19_region_le_total`	`propext`	Encoded region-versus-total inequality
F22	`f22_khipu_emit_monotone`	`propext`, `Quot.sound`	`List.range` sequence-log monotonicity
F22	`f22_emit_appends_length`	`propext`	Encoded append/length property
F22	`f22_emit_strictly_greater`	`propext`, `Quot.sound`	Encoded strict-growth property

The table demonstrates why names and interpretations must be separated. "Kuramoto," "Reed-Solomon," and "Bekenstein" evoke substantial scientific theories. The admitted local declarations are smaller arithmetic, list, or order statements. The proof receipt supports those statements. It does not transfer automatically to nonlinear dynamics, a decoder implementation, or a physical entropy bound.

7.4 Served registry

The served A11oy guard reports seven declaration-backed entries. Audit of their actual frontier yields Table 6.

Table 6. Served registry boundary.

Entry	Guard result	Audit disposition	Required promotion evidence
`pacbayes`	declaration-backed	supported	Isolated compile and axiom report for `pacBayes_inequality_form`; avoid inheriting two wrapper placeholders
`welford`	declaration-backed	supported	Isolated compile, placeholder scan, and axiom report
`quorum`	declaration-backed	open conjecture	Close or narrow `khipu_consensus_safety`, which contains `sorry` and is labeled Conjecture 2
`bloom`	declaration-backed	supported	Isolated compile and axiom receipt for `query_after_insert`
`kalman`	declaration-backed	supported	Isolated compile and axiom receipt for `posterior_le_prior`
`bls`	declaration-backed	supported model	Preserve abstract additive-module boundary; no production BLS security claim
`hnsw`	declaration-backed	supported model	Preserve strictly-decreasing-distance model; no general deployed HNSW performance claim
eight other entries	experimental	experimental or measured as applicable	Exact artifact, commit, receipt, and empirical evidence where required

The eight experimental entries are `holevo`, `reidemeister`, `kl`, `pinsker`, `aftershock`, `allodial`, `entanglement`, and `sovereign`. Their presence in a served index provides no proof credit. `aftershock`, in particular, would require empirical evidence; Lean could prove properties of an analysis method but not create the observations.

7.5 Proof-obligation queue

Table 7 gives both task and disposition partitions.

Table 7. Proof-obligation queue.

Partition	Class	Count	Meaning
Disposition	OPEN	200	Work remains; truth and tractability are not assumed
Disposition	REFUTED	3	Exact targets have negative evidence and must not be promoted
Task	FORMALIZE_AND_PROVE	131	Specify exact statements, test for counterexamples, then attempt proof
Task	REPAIR_EXACT_LEAN_BINDING	63	Resolve current module, declaration, type, and source identity
Task	DISCHARGE_OR_MEASURE	7	Supply named empirical data or measurement protocol
Task	PRESERVE_COUNTEREXAMPLE	2	Keep negative witnesses executable and attached to targets

The task counts sum to 203, as do the disposition counts. The two partitions answer different questions, so their rows are not summed together. The three refuted dispositions and two counterexample-preservation tasks need not be equal because task class and current disposition are independent fields.

7.6 Conditional source-present artifacts

The audit identified useful Lean source outside the locked set. The strengthened Lambda theorem discussed above is source-present and conditional on factorization. Two Ouroboros modules specialize fixed-point results: one invokes a contraction fixed-point theorem under completeness and an explicit contraction factor; the other gives a least fixed point in a complete-lattice/order-homomorphism model. Local inspection found no `sorry` in their theorem bodies, and the whole library build is green. However, the available evidence does not include dedicated per-declaration axiom receipts for these statements. Their status is therefore `SUPPORTED_SOURCE_PRESENT`, not part of the locked nineteen.

The boundary matters. A Banach-style theorem under a contraction hypothesis does not prove that an arbitrary self-revision loop contracts, terminates, or is safe. A least-fixed-point theorem in an order model does not establish wall-clock convergence or safe self-modification. A future formal account should separate progress or fuel, invariant preservation, and conditional convergence into distinct obligations.

7.7 Research-question answers

The product model answers RQ1 by preserving the exact 8/19, 15, 21, 30, 80, 179, and 203 partitions without a universal sum. For RQ2, a whole build requires statement binding, content identity, closure analysis, per-declaration acceptance, axiom output, an allowlist, and an interpretation boundary before proof admission. RQ3 is answered by persistent refutation records and `COLLISION` states that transfer no evidence. For RQ4, a reviewable release still needs immutable sources and environment, per-declaration and closure reports, a semantic crosswalk, count scripts, licenses, and independent reproduction.

8. Threats to validity and limitations

8.1 Construct validity

"Proof" has multiple philosophical and technical meanings. This study operationalizes it as an admission decision under a pinned Lean environment and an explicit local policy. That construct is appropriate for repository governance but does not establish empirical truth, model adequacy, security, or correctness of every imported foundation. We mitigate this threat by using `PROVED_WITHIN_SCOPE` and by reporting axiom surfaces and interpretation boundaries.

The term "source closure" is also easy to overstate. A zero-placeholder scan of one target module is weaker than a transitive analysis of the theorem's dependencies. The current selected receipt reports zero non-comment `sorry` in its module and printed axiom sets for declarations. Our abstract policy asks for dependency-closure evidence; a public release must implement that stronger check rather than implying it was already performed.

8.2 Internal validity

The audit was conducted by one primary analyst with AI assistance. Parser mistakes or undocumented generation rules could affect counts. One concrete hazard was that the queue uses `disposition`, not `status`; totals were regenerated from the actual field, and hashes and byte lengths are reported. Independent reproduction remains required.

The selected receipt is a local JSON artifact, not an externally signed attestation. Its command and outputs should be rerun from a content-addressed environment. The container tag recorded in the receipt is mutable. The full build uses a fresh Ubuntu container and official `elan`, but no immutable container image digest is present in the artifact. These omissions limit supply-chain assurance.

8.3 External validity

This single corpus has project-specific labels and policies. Other proof assistants and scientific repositories require adapted state machines and allowlists. Its counts do not estimate error rates in formalization repositories generally, and the proposed schema cannot by itself eliminate overclaiming.

8.4 Conclusion validity

No hypothesis test, effect size, or confidence interval is reported because this is a deterministic artifact audit, not a sampled population study. The reported counts are exact only for the hashed inputs. They should not be interpreted as stable project metrics after the sources change.

No proof-generation model was trained or evaluated. The zero accepted-trajectory result means that benchmark comparisons, learning curves, contamination analysis, and proof-search success rates are out of scope. Any claim that the queue improves model performance would require a separate preregistered experiment and held-out evaluation.

8.5 Semantic and historical risks

Names such as "Bekenstein," "Kuramoto," "Reed-Solomon," and culturally specific terms can invite claims broader than the encoded mathematics. Historical or cultural provenance can motivate formal structures but is not itself proof evidence. Attribution should be reviewed by subject-matter experts, and community or historical claims should not be inferred from identifier names.

The audit found no exact formal artifact for one historically discussed name in the broader workspace, "Alabada" or "Alabadda." It remains unspecified and is excluded from all proof totals. This illustrates the fail-closed policy for uncertain spelling, provenance, equation, and semantics.

8.6 Reproducibility limitations

The accompanying archive uses archive-relative paths for bundled material and `${WORKSPACE_ROOT}/...` only for nonbundled workspace sources. No repository or dataset DOI has been minted. License and redistribution status are consolidated in `RIGHTS_AND_PROVENANCE.md`, with full applicable CC BY 4.0 and Apache 2.0 texts. The canonical Lake source expected in a related operational contract was unavailable: the observed `szl-lake` material was a 52-file dataset snapshot with no Lean source or compiled Mathlib assets. Therefore no claim in this paper depends on a canonical Lake formula or on that snapshot as a proof source.

The audit date and source pins are explicit, but bit-for-bit environment reproduction also needs the container image digest, platform packages, full dependency closure, and complete archived build logs. The present archive is externally distributable as a frozen preprint evidence package, but it is not a self-contained execution environment and does not establish independent reproduction.

9. Related work

9.1 Formal proof infrastructure

Lean 4's extensible prover/programming-language design and Mathlib's distributed formal library provide the technical substrate for the corpus [1, 2]. The present work does not modify Lean's trusted kernel or propose a new logic. It operates one level above: it governs how organizations bind informal claims and inventory records to Lean artifacts, toolchains, placeholder scans, and axiom policies.

Large formal libraries have long distinguished proof terms from source presentation, but mixed organizational repositories introduce additional evidence classes that the kernel cannot resolve. The kernel can check an encoded theorem; it cannot determine whether a public formula identifier refers to that theorem, whether an implementation conforms to it, whether an empirical observation occurred, or whether a marketing interpretation exceeds the type. Our schema makes these bridge obligations explicit.

9.2 Theorem-proving datasets and models

LeanDojo extracts proof states and premise annotations from Lean and supports programmatic interaction and retrieval-augmented proving [5]. Its concern with precise accessible premises reinforces the importance of stable, qualified identities. miniF2F supplies a cross-system benchmark of 488 formal problem statements drawn from olympiad and educational sources [6]. PACT extracts self-supervised learning signals from proof artifacts and reports an improvement in its evaluated theorem-proving setting [7]. These works demonstrate the value of well-defined proof corpora and evaluation units.

Our contribution is complementary. We do not introduce a theorem-proving model or benchmark score. We address admission before training: a manuscript block, executable formula, declaration presence record, and refuted conjecture should not be flattened into positive proof trajectories. This is particularly important for real repositories, where tasks are embedded in large dependency graphs and can include open placeholders. SorryDB explicitly constructs a dynamically updated benchmark of open Lean tasks from real-world projects and evaluates a fixed snapshot [8]. Our 203-record queue resembles such work only at a high level; it has zero accepted trajectories and includes binding, empirical, and counterexample-preservation tasks that are not all theorem-completion problems.

The distinction affects contamination and evaluation. A false target should not lower a prover's apparent ability because no valid proof exists under the stated assumptions. A binding repair should not be scored as a theorem. An empirical task should not enter a tactic benchmark. Evidence typing is therefore a preprocessing requirement for trustworthy proof datasets.

9.3 Dataset and research-artifact documentation

Datasheets for Datasets propose documenting motivation, composition, collection process, uses, and limitations [4]. The FAIR principles emphasize findability, accessibility, interoperability, and reusability for data and related research objects, including provenance and licensing [3]. We specialize these documentation ideas to proof-bearing mixed corpora. A statement digest, theorem binding, toolchain, axiom surface, counterexample, and claim boundary are analogous to dataset composition and recommended-use fields, but are tailored to formal admission.

FAIRness and proof are independent. A perfectly documented open obligation can be FAIR without being proved. A kernel-accepted theorem stored on a private machine can be proved within its formal environment without being findable or reusable. A publication artifact should seek both, but one status must not substitute for the other.

9.4 Characterizations of means

The Lambda case belongs to a substantial functional-equation literature, not to a new theorem established here. Aczel and Saaty investigate conditions for synthesis functions, explicitly including separability, associativity or bisymmetry, cancellativity, consensus, reciprocal properties, and homogeneity [10]. Csato characterizes a row-geometric-mean ranking in a pairwise-comparison setting using an explicit axiom set including aggregation invariance [11]. Recent work studies quasi-arithmetic means and the role of bisymmetry, reflexivity, symmetry, strict increase, compatibility, and regularity [9, 12].

The relevant methodological lesson is that uniqueness is assumption-sensitive. The local A1-A5 collection is not interchangeable with the hypotheses of these characterization results. The `maxAgg` witness shows the gap directly. Citing the literature cannot fill a missing Lean hypothesis; it helps identify a plausible strengthened statement that must then be formalized and receipted on its own terms.

9.5 Artifact evaluation and proof governance

Proof artifacts extend ordinary reproducible-software records with theorem identity, dependency and axiom surfaces, and an admission rule. Our model derives proof status per declaration and makes no supply-chain attestation claim for its unsigned local receipts.

10. Reproducibility protocol

10.1 Minimal reproduction

A reviewer with the archived sources should perform the following steps:

1. Verify every artifact digest in Appendix A before parsing.
2. Check out Lutar-Lean commit `85dee499b72cc71f49907c70e62018ab85dba01c` and verify Mathlib commit `aa936c36e8484abd300577139faf8e945850831a`.
3. Install Lean `v4.18.0` in a fresh, content-addressed Linux environment.
4. Run `lake exe cache get` and `lake build`; archive complete standard output, standard error, exit code, and timestamps.
5. Run the two guard-test files recorded in the full-build receipt and confirm 21 pass and zero fail.
6. Verify SHA-256 `acce5d750702cff89c0f84a9cc0a5d7ff22264d76d5229e1b47d3613b7ab77a1` for `Lutar/Puriq/Formulas/ProvedFormulas.lean`.
7. Compile the selected module and scan the declared scope for non-comment `sorry` and `admit`.
8. Print and store the axiom set for each of the nineteen declarations; compare it to `propext` and `Quot.sound` as permitted by the receipt policy.
9. Recompute every inventory and queue partition from the frozen inputs.
10. Compile and execute the Lambda counterexample; verify the `(4,1)` witness and retain the false unconditional target as REFUTED.
11. Run the formula crosswalk validator once the pre-submission crosswalk exists; confirm that all eight `F*` collisions fail label-only joins.

Any mismatch should be reported by coordinate. For example, a changed file digest is `IDENTITY_MISMATCH`; a missing declaration is `BINDING_FAILURE`; an unexpected axiom is `AXIOM_POLICY_FAIL`; and a different total with matching input hashes suggests a parser or grouping defect. A single generic `FAIL` would discard useful diagnostic evidence.

10.2 Release contents and reproducibility limits

The accompanying preprint package contains:

- the path-sanitized formula-readiness ledger and exact proof-obligation JSON Lines file;
- exact full-build and selected-declaration receipts;
- the exact selected Lean module named by the receipt;
- the semantic formula-ID crosswalk and standalone validator;
- machine-readable release, checksum, and transformation manifests; and
- `RIGHTS_AND_PROVENANCE.md` with per-source-family redistribution status.

The package does not contain the selected module's complete dependency closure, a content-addressed container image, complete build logs, or an independent replay. Those remain reproducibility limitations, not reasons to withhold the accurately labeled preprint. No DOI or release URL is supplied because none exists; artifacts are identified by archive-relative path and digest.

10.3 Promotion checklist

A reviewer promoting any new declaration should be able to answer "yes" to all of the following:

- Is the natural-language statement exact and no broader than the theorem type?
- Are the domain, codomain, hypotheses, and modeled interpretation explicit?
- Is the fully qualified Lean name bound to a commit and file digest?
- Does the target and its admitted closure contain zero unresolved placeholders under the scan policy?
- Does the pinned Lean environment accept the declaration?
- Is the printed axiom set a subset of a reviewed, versioned allowlist?
- Has a counterexample search appropriate to the domain been retained?
- Are tests and empirical results labeled separately from formal proof?
- Is any alias collision resolved by a semantic crosswalk?
- Is the receipt signed, or plainly labeled as an unsigned local receipt?
- Has another person or team independently reproduced the result?

The archived nineteen-declaration set satisfies the existing receipt's narrower checks and is reported within that scope. The accompanying package supplies immutable checksummed bytes; independent reproduction and authenticated publication remain open for the manuscript as a whole.

11. Ethics, governance, and AI-assistance disclosure

11.1 Research ethics and foreseeable misuse

This artifact audit involved no recruited participants, intervention, clinical data, or intended analysis of personal behavior. It examined local source code, structured metadata, receipts, and manuscript artifacts. Nevertheless, a public release requires a scan for secrets, personal information in logs or commit metadata, third-party licensing obligations, and culturally or historically sensitive attributions. Absence of a human-subject experiment is not a substitute for responsible data release.

The principal foreseeable misuse is overclaiming. A proof label can lend unwarranted authority to an empirical, security, physical, or governance assertion. Names such as BLS, HNSW, Kuramoto, Reed-Solomon, and Bekenstein

may make a small model theorem appear to validate a complete deployed system or scientific law. This preprint mitigates that risk by including claim boundaries in the primary results table and by assigning zero formal credit to experimental or declaration-presence states.

Refutation artifacts also require care. They should be used to correct specifications, not to disparage contributors who formulated an incomplete conjecture. Formalization is iterative, and an explicit counterexample is a valuable research outcome. Authorship and contributor roles must be established from documented intellectual contributions rather than inferred from commit trailers, tool output, or automation identities.

11.2 AI assistance

An OpenAI Codex system assisted with local artifact inspection, deterministic count queries, bibliography discovery restricted to local evidence and primary sources, structural editing, and drafting of this manuscript. The system did not receive authorship and should not be cited as an author. The named human author remains responsible for verifying every claim, theorem boundary, citation, license, and disclosure before dissemination.

AI-generated text and analysis were not accepted as proof evidence. The nineteen-declaration claim comes from the hashed Lean receipt and must be independently rerun. The Lambda refutation comes from inspected Lean source and likewise requires reproduction. No model-generated proof was promoted, no accepted proof trajectory was produced, and no model-performance result is reported. Bibliographic metadata was checked against official Lean documentation, publisher pages, or primary arXiv records; the final reference list should receive conventional human bibliographic review.

11.3 Governance of future automation

Future agents operating on the queue should be constrained by the evidence schema. They may propose statements, counterexamples, bindings, or proof terms, but only deterministic validators may derive admission state. Agents must not rewrite REFUTED to OPEN, infer theorem bindings from alias similarity, convert measurement tasks into formal tasks, or describe unsigned digests as signatures. Human review is required for interpretation boundaries, axiom policy changes, licensing, historical attribution, and publication claims.

12. Discussion

The audit's most important result is not the number nineteen. It is that nineteen remains stable when much larger numbers appear nearby. A robust evidence system should make the conservative interpretation easier than the expansive one. Here, the whole build's 13,308 targets, current source's 1,401 declarations, manuscript's 179 blocks, and queue's 203 records are all true counts in their respective scopes. The schema prevents any of them from overwriting the selected receipt.

This suggests that proof governance should be designed as a data-integration layer. The hard problems are not only tactics and theorem statements; they include identity, lineage, semantic joins, policy, and negative records. The `c7/85dee499` mismatch would be difficult to detect from a theorem name alone. The `F*` collision would be invisible in separate tables. The Lambda issue would look like a stubborn open goal without counterexample-aware classification. Each is resolved by preserving more structure, not by increasing a confidence score.

Formal proofs, tests, measurements, and model-to-implementation conformance are complementary receipts; stronger systems claims require explicit bridges. Maintaining per-declaration reports, closure scans, digests, and crosswalks has a cost and produces smaller public totals, but it turns binding repair, counterexample preservation, environment pinning, and honest claim narrowing into visible progress.

13. Conclusion

Mixed formal and executable formula corpora cannot be governed by a single verification flag. Artifact identity, statement binding, source closure, kernel acceptance, axiom policy, tests, empirical evidence, and refutation are distinct coordinates. Applying that separation to the audited corpus yields a precise result: eight locked formula families contain nineteen receipt-backed declarations; the other inventories retain their own scoped counts; and the 203-record work queue remains 200 open and three refuted.

A successful 13,308-target build is important integration evidence, not a theorem total. The Lambda counterexample shows why negative evidence must be preserved and why added assumptions create a new statement rather than retroactively proving a false one. Formula-ID collisions show why names cannot substitute for semantic bindings.

The manuscript's contribution is a governance method, an exact reconciliation, and a reproducible promotion protocol. It establishes no new characterization theorem and reports no proof-generation experiment. The accompanying archive now includes immutable selected sources, exact eligible receipts, documented path-sanitized derivatives, licenses, a semantic crosswalk, and standalone validation. Full dependency closure, a content-addressed execution environment, fresh per-declaration replay, and independent reproduction remain future work. This document is an unreleased, externally distributable preprint; it is not peer reviewed, deposited, or represented as submission-ready.

Appendix A. Artifact manifest and hashes

All hashes use SHA-256 over the exact local bytes observed during drafting.

Artifact	Bytes	SHA-256
Formula/proof frontier audit	14,709	`d6d7aa314d3983e527597e979f2ef9aa3a3b3275dc0fbd49afe15c1756ce72c9`
Full Lutar-Lean build receipt	1,054	`50af3d31ec20efc9a696a71130958c653e5fb125174567ac6cebf66bd954b875`
Eight-family selected-declaration receipt	3,239	`7ace9bf5d7cdbad19ec4f9c17dc44ff24697050e96dd245713faf8f83407d25f`
Proof-obligation queue v1	106,642	`fc3442050651472550 93a6d947749abc9a20f1251c02c461498291cb8cc20952`
Formula-readiness ledger	158,697	`a3bed18056859153475f3bb8ca7263fa8439680e2555e95f7eacf907d66ebf55`
Lake/Mathlib/Octave operational contract	1,916	`589a466436558ad3ca5f58d9a6abd9c2f7f0d522da6df7e7753cd3ba4e806583`
Publication portfolio roadmap (Markdown)	19,966	`28b5604bbbaf2fc0750ad8b071d43f073f7138a2f7fa73c51b1fa8c8a5c2659c`
Publication portfolio roadmap (JSON)	11,031	`4fc4ee1a57a083576341862cb8b666576052b9d05d67396b524e9356fd6f5fbb`
Selected `ProvedFormulas.lean` source, as recorded by its receipt	not recomputed in this manuscript manifest	`acce5d750702cff89c0f84a9cc0a5d7ff22264d76d5229e1b47d3613b7ab77a1`

The hash of this manuscript is intentionally not embedded here because doing so would be self-referential. It must be reported in the release receipt and alongside the final file.

Appendix B. Quarantined and prohibited claims

The following statements are not supported by the audited evidence and must not appear in an abstract, title, highlight, metadata record, or public status card:

- "180 proved," "200 proved," "all formulas proved," or a sum across Table 3 inventories;

- "13,308 theorems proved"; the receipt says build targets completed;
- "203 proof trajectories"; the records are obligations and accepted trajectories equal zero;
- "Lambda uniqueness proved" without the exact strengthened assumptions and a dedicated receipt;
- "Ouroboros termination, convergence, or safe self-improvement proved" without the theorem's explicit hypotheses and separate progress/safety receipts;
- "Reed-Solomon decoder verified," "Kuramoto synchronization proved," or "physical Bekenstein bound proved" from the smaller encoded declarations in Table 5;
- "production BLS security" or "general HNSW performance" from the supported-model sources;
- evidence transfer between colliding `F*` labels without the semantic crosswalk;
- proof attribution to historical pin `c7c0ba17...` for a module absent from that pin;
- "signed receipt" for deterministic-digest-only or unsigned local artifacts;
- "published," "peer reviewed," "accepted," "deployed," or "independently reproduced" for this preprint.

Appendix C. Suggested crosswalk schema

```
{
  "schema": "szl.formula-crosswalk/v1",
  "global_id": "<stable content-independent identifier>",
  "aliases": [{"namespace": "brain", "value": "F1"}],
  "statement": "<normalized mathematical statement>",
  "domain": "<exact domain>",
  "codomain": "<exact codomain>",
  "hypotheses": ["<explicit hypothesis>"],
  "interpretation_boundary": "<what the statement does not establish>",
  "lean": {
    "repository": "<repository>",
    "commit": "<40-hex commit>",
    "module": "<module>",
    "declaration": "<fully qualified name>",
    "source_sha256": "<64-hex digest>"
  },
  "receipt_sha256": "<64-hex digest>",
  "evidence_state": "<typed state>",
  "collision_state": "CLEAR | COLLISION | UNRESOLVED",
  "review": {
    "signature_state": "SIGNED | UNSIGNED_LOCAL",
    "reviewer": null,
    "reviewed_at": null
  }
}
```

The global identifier must not be the reused `F*` alias. A normalized-statement digest can assist deduplication but should not replace a readable statement and review, because syntactically different statements may be equivalent and syntactically similar statements may differ in hypotheses.

References

[1] Leonardo de Moura and Sebastian Ullrich. "The Lean 4 Theorem Prover and Programming Language." *Automated Deduction-CADE 28*, Lecture Notes in Computer Science, 2021, pp. 625-635. doi:10.1007/978-3-030-79876-5_37.

[2] The mathlib Community. "The Lean mathematical library." arXiv:1910.09336, 2019.

[3] Mark D. Wilkinson et al. "The FAIR Guiding Principles for scientific data management and stewardship." *Scientific Data* 3, 160018, 2016. doi:10.1038/sdata.2016.18.

[4] Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daume III, and Kate Crawford. "Datasheets for Datasets." arXiv:1803.09010, 2018.

[5] Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. "LeanDojo: Theorem Proving with Retrieval-Augmented Language Models." arXiv:2306.15626, 2023.

[6] Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. "MiniF2F: a cross-system benchmark for formal Olympiad-level mathematics." arXiv:2109.00110, 2021.

[7] Jesse Michael Han, Jason Rute, Yuhuai Wu, Edward W. Ayers, and Stanislas Polu. "Proof Artifact Co-training for Theorem Proving with Language Models." arXiv:2102.06203, 2021.

[8] Austin Letson et al. "SorryDB: Can AI Provers Complete Real-World Lean Theorems?." arXiv:2603.02668, 2026.

[9] Pal Burai, Gergely Kiss, and Patricia Szokol. "Characterization of quasi-arithmetic means without regularity condition." arXiv:2107.07391, 2021.

[10] Janos Aczel and Thomas L. Saaty. "Procedures for synthesizing ratio judgements90028-7)." *Journal of Mathematical Psychology* 27(1), 1983, pp. 93-102. doi:10.1016/0022-2496(83)90028-7.

[11] Laszlo Csato. "Characterization of the row geometric mean ranking with a group consensus axiom." arXiv:1706.07256, 2017; later published in *Group Decision and Negotiation* 27(6), 2018, pp. 1011-1027. doi:10.1007/s10726-018-9589-3.

[12] Gergely Kiss and Ekaterina Shulman. "N-ary quasi-arithmetic means and families without regularity." arXiv:2606.05221, 2026.